SWE 402 - Game Programming

Assignment 2: Choose your game

Choose **ONE** option: **2D Roguelike Game** or **3D Stealth Game: Haunted House**. Complete the requirements for your chosen option. You do not need to build both games.

Deadline and submission - both options

Due: Monday, October 19, 2026 at 2:00 PM Riyadh time (UTC+3).

Submit your **GitHub repository link on Blackboard**. Include the complete Unity project and a proper README.md stating which option you chose, how to open and run the project, and any bonus features implemented. Include the build or published-game information required by your chosen option.

The Roguelike requirements, platform builds, and bonus-point rules below apply to **Option 1 only**. For Option 2, follow the Haunted House course and the checklist in its own section. The deadline and GitHub submission above apply to both options.

Option 1: 2D Roguelike Game

Build a turn-based roguelike with procedural levels, enemies, and resource management. Complete the Unity Learn course and the additional requirements on the following pages.

Unity Learn: Create a 2D Roguelike Game

Option 2: 3D Stealth Game: Haunted House

Build a playable 3D stealth game by completing the Haunted House course. Its checklist appears after the Roguelike requirements.

Unity Learn: 3D Stealth Game: Haunted House

Option 1 requirements: 2D Roguelike Game

Section 1: Project Setup & Game Board

Complete: "Set up your project" + "Add a game board"

- Create a new Unity project using Editor 6.x LTS and the Universal 2D template
- Import the provided Roguelike2D tutorial assets package
- Set the Main Camera to Orthographic, size 5, solid black background
- Create a Tile Palette with ground tiles from the provided Sprite Sheets
- Write a BoardManager script that procedurally generates an 8×8 tilemap using nested for-loops
- Use Tile[] arrays and Random.Range() to randomly assign ground and wall tiles
- Add border detection logic so edge cells use wall tiles
- Create a CellData nested class with a Passable boolean; store board state in a CellData[,] 2D array
- Set up Unity Version Control and check in your initial changes

Section 2: Player Character & Movement

Complete: "Add a player character"

- Create a PlayerCharacter GameObject from a provided sprite; set Order in Layer for correct rendering
- Write a PlayerController script that reads arrow key input via Input.GetKeyDown()
- Implement grid-based (cell-to-cell) movement with passability checks using GetCellData()
- Create a Spawn() method to initialize the player at a given cell coordinate
- Refactor BoardManager: rename Start() to Init() and make it public for external initialization

Section 3: Turn System & Game Architecture

Complete: "Add a turn system"

- Create a TurnManager as a pure C# class (non-MonoBehaviour) with a constructor and Tick() method
- Create a GameManager MonoBehaviour that initializes TurnManager, BoardManager, and PlayerController
- Implement the Singleton pattern on GameManager using static Instance with public get; private set;
- Use Awake() for singleton enforcement and Start() for game initialization
- Call TurnManager.Tick() from PlayerController each time the player moves

Section 4: Food Resource & Inheritance

Complete: "Add a food resource"

- Create a CellObject base class with a virtual PlayerEntered() method
- Create a FoodObject subclass that overrides PlayerEntered() to grant food and destroy itself
- Store a ContainedObject reference in CellData so cells know what objects they hold
- Add food tracking to GameManager; decrease food each turn, increase on collection
- Use UI Toolkit (UI Builder window) to create a FoodLabel that displays the current food count
- Create multiple food prefabs with different AmountGranted values; randomize spawns at level start

Section 5: Obstacles, Win/Lose Conditions & Levels

Complete: "Add obstacles" + "Add win and lose conditions"

- Create a WallObject subclass of CellObject that changes the tile sprite and blocks player movement
- Implement PlayerWantsToEnter() pattern — walls take damage on bump and are destroyed at 0 HP
- Create an ExitCellObject subclass that triggers level completion when the player enters its cell
- Implement level progression: clear the board and regenerate a new level on exit
- Add a Game Over state when food reaches 0 — disable input and display a Game Over panel via UI Toolkit
- Implement game restart functionality on key press to return to level 1

Section 6: Animation & Enemies

Complete: "Animation" + "Add an enemy"

- Add an Animator component to the player with an AnimatorController containing Idle, Walk, and Attack states
- Create sprite-based keyframe animations using the Animation window; set sample rate appropriately
- Configure state transitions with proper Has Exit Time, Transition Duration = 0, and Bool/Trigger conditions
- Control animations from code using SetBool() and SetTrigger()
- Create an Enemy class inheriting from CellObject that subscribes to TurnManager.OnTick using += and unsubscribes in OnDestroy() using -=
- Implement enemy AI: move toward the player each turn; attack (reduce food) if adjacent
- Enemy blocks player entry, takes damage on bump, and is destroyed at 0 HP (same pattern as WallObject)

Section 7: Additional Requirements (Beyond the Course)

After completing all course tutorials, implement all of the following enhancements to your project. These integrate concepts from previous missions (M1–M6) that the Roguelike course does not cover.

Audio (M3 concepts)

- Add background music via an AudioSource on the Main Camera (loop enabled, reduced volume)
- Add sound effects using PlayOneShot() for: player movement, wall attack, food pickup, enemy attack, enemy death, and game over

Visual Effects (M3 concepts)

- Add ParticleSystem effects for at least 3 events: wall destruction, food collection, and enemy death
- Control particles from code using .Play() and .Stop(); instantiate or pool particle prefabs as needed

Smooth Cell Movement (M3–M5 concepts)

- Replace instant grid snapping with smooth movement using a Coroutine that lerps the player between cells
- Block input during the movement coroutine to prevent skipping cells

Object Pooling (M6 concepts)

- Implement object pooling for CellObjects (food, walls, enemies) — pre-instantiate a pool at game start
- On level transition, recycle objects via SetActive(false/true) instead of Destroy/Instantiate

Code Standards (M6 concepts)

- Use [SerializeField] on all private variables exposed to the Inspector — no unnecessary public fields

- Use [Range()] on numeric tuning variables (e.g., speeds, spawn counts) and [Tooltip()] where helpful

Section 8: Build & Submit

Build your completed project for all three platforms and submit.

- Install Windows, macOS, and WebGL build modules via Unity Hub

- Add your scene(s) to File → Build Profiles → Scene List

- Build for Windows (standalone .exe), macOS (.app), and WebGL (index.html + supporting files)

- Test each build to verify it runs correctly outside the Editor

- (Optional) Upload your WebGL build to Unity Play or itch.io for browser testing

Submission: Use the shared GitHub submission instructions on page 1. Include the Builds/ folder with Windows, macOS, and WebGL builds.

Bonus Features (Optional — Up to 10 Extra Points)

Implement any of the following for extra credit. Each feature is worth up to 5 points.

- Dynamic Board Scaling: Increase board size and/or number of enemies/walls as the player progresses to higher levels

- Multiple Enemy Types: Create 2+ enemy subclasses with different AI behaviors (e.g., one chases, one patrols, one ambushes)

- Items & Stats System: Add collectible items (weapons, armor, potions) that modify player stats using ScriptableObjects

- Save & Load System: Implement data persistence across sessions using JSON serialization (PlayerPrefs or file I/O)

- Main Menu & Pause Menu: Add a title screen with Play/Quit buttons and an in-game pause menu with Resume/Restart/Quit

- Custom Sprites & Theme: Replace all provided assets with your own sprites or a cohesive custom theme

Resources

- Unity Learn Course: Create a 2D Roguelike Game

- Unity 6 Documentation

- Unity Manual: Tilemaps

- Unity Manual: UI Toolkit

- Unity Learn: Introduction to Object Pooling

- Free Assets: Unity Asset Store, OpenGameArt.org, Kenney.nl

Option 2 requirements: 3D Stealth Game: Haunted House

Complete the course tutorials and build your own working Haunted House game. Use the linked Unity Learn course as the detailed step-by-step reference.

Open the Haunted House course

1. Scene setup

Set up the game scene using the course assets and arrange the haunted-house environment.

2. Player movement

Implement player controls so the character can move through the environment.

3. Player animation

Animate the player and connect animations to movement.

4. Camera setup

Configure the camera to show and follow gameplay as taught in the course.

5. Static enemies

Create the static enemies and implement their detection behavior.

6. Dynamic enemies

Create moving enemies and implement their movement and detection behavior.

7. Ending the game

Implement the game ending, including successful escape and being caught.

8. Audio

Add the audio taught in the course and verify it during gameplay.

9. Publish and share

Complete the course publishing and sharing exercise; include the resulting build or published-game link in your README.

Check and submit

Test movement, animation, camera behavior, enemy detection, win/lose behavior, and audio. Submit the complete Unity project through your GitHub repository link on Blackboard, with a proper README.md identifying Option 2 and explaining how to run the game.

The course Bonus features tutorial is available for optional extensions. The Option 1 bonus-point scheme does not apply to this option.